

CeTTa-Hosted Supercompilation for MeTTa: Comprehensive Feedback and Brainstorming Summary

Compiled from discussion by
Ben Goertzel, ProtocosmoBot (ZeroBot/OpenClaw),
ProtomegaBot (OmegaClaw), and Oruzibot

July 5–6, 2026

Abstract

This document compiles all feedback, suggestions, and brainstorming from the Telegram group discussion regarding Ben Goertzel’s CeTTa-Hosted Supercompilation proposal (July 5, 2026) and its predecessor document, A MORK-Native Supercompiler for MeTTa+MM2 (October 20, 2025). The discussion involved Ben Goertzel, ProtocosmoBot (ZeroBot/OpenClaw), ProtomegaBot (OmegaClaw), and Oruzibot. Feedback is organized by contributor, then by theme: architectural strengths, specific technical concerns, cross-document comparisons, and connections to broader system architecture.

Contents

1	Source Documents	3
2	ProtocosmoBot (ZeroBot/OpenClaw) Feedback on the CeTTa Proposal	3
2.1	Strengths Identified	3
2.2	Technical Concerns and Suggestions	3
2.2.1	Tensor Presentation Is Under-Specified	3
2.2.2	Effect System Is the Hidden Critical Path	4
2.2.3	Hot-Region Detection Is Unspecified	4
2.2.4	Space Revision Invalidation Under Concurrent Mutation	4
2.2.5	Certificate Layer Risks Becoming Documentation Rather Than Verification	4
2.2.6	Missing Anti-Pattern: Space Schema Stability Across Profile Upgrades	4
2.3	Overall Assessment	4
3	ProtomegaBot (OmegaClaw) Feedback on the CeTTa Proposal	5
3.1	Strengths Identified	5
3.2	Technical Concerns and Suggestions	5
3.2.1	Space Revision Policy Needs More Specification	5
3.2.2	Certificate Layer Risks Becoming a Bottleneck	5
3.2.3	Bag Multiplicity Handling Needs a Concrete Protocol	5
3.2.4	Local Search Episode Is Underspecified Relative to Its Ambition	5
3.2.5	No Mention of GPU Lowering Path for Tensor Presentation	6
3.2.6	Benchmarking Discipline	6
3.3	Connection to OmegaHive 1.1	6

3.4	Additional Observations After Reviewing the v2 Background Document	6
4	Cross-Document Comparison: CeTTa vs. MORK-Native Supercompiler v2	7
4.1	What CeTTa Preserves from the v2 Document	7
4.2	What CeTTa Changes or Supersedes	7
4.3	What the v2 Document Has That CeTTa Could Use	8
4.4	Overall Assessment of the Pair	8
5	Consolidated Suggestion List	8
5.1	Specification Gaps to Fill	8
5.2	Architecture and Design Suggestions	9
5.3	Verification and Certificate Suggestions	9
5.4	Benchmarking and Testing Suggestions	10
5.5	Integration and Sharing Suggestions	10
6	Participant Attribution Summary	10
7	Recommended Next Steps	10

1 Source Documents

1. **CeTTa-Hosted Supercompilation for MeTTa: A Human-Oriented Architecture with a Coding-Agent Appendix** (Ben Goertzel, July 5, 2026). Proposes a synthesis of CeTTa, LeaTTa/Lean, MORK/PathMap, MM2, and a multi-geometry intermediate representation for aggressive MeTTa supercompilation. The central thesis: use LeaTTa/Lean for semantic checking, use the multi-geometry IR to choose computational presentation, and use CeTTa as the guarded runtime host.
2. **A MORK-Native Supercompiler for MeTTa+MM2: Formal Design with Minimal Core and Query Planning** (Ben Goertzel, October 20, 2025). Earlier design featuring a minimal core IR (M-Core), comprehensive effect algebra, query planner for pattern-rewriting semantics, statistics-guided splitting, canonical keys for termination, and bisimulation-based correctness proofs. This document provides the formal foundations that the CeTTa proposal builds on.

2 ProtocosmoBot (ZeroBot/OpenClaw) Feedback on the CeTTa Proposal

2.1 Strengths Identified

1. **Multi-geometry IR with deferred lowering** is the single most important architectural decision. Choosing the presentation *after* seeing profile data and effect analysis — rather than committing to MM2 or C upfront — is clearly right. Early lowering is the classic supercompiler mistake.
2. **PathMap narrows, native rematch decides.** This boundary is conservative, correct, and avoids the seductive trap of treating trie keys as binding structures. The explicit call-out of bag multiplicity as separate metadata is exactly the kind of semantic trap that would silently corrupt results if left implicit.
3. **CettaPack with manifest guards + fallback** makes aggressive optimization *safe by construction*. A failed guard degrades to correct behavior rather than producing wrong answers. This is the right safety architecture.
4. **The four optimization patterns are correctly sequenced.** Match compilation (Pattern 1) is the natural MVP — it is the most common hot path and exercises the key machinery (PathMap candidates, native rematch, C kernel generation, CettaPack loading) without requiring the harder patterns.

2.2 Technical Concerns and Suggestions

2.2.1 Tensor Presentation Is Under-Specified

The jump from MeTTa semantics to semiring contractions / GPU kernels is non-trivial. What is the concrete lowering story? For PLN truth values to tensor, the TV algebra must map cleanly to semiring operations, and the active-subgraph extraction (Pattern 3) must produce tensor-shaped data. This deserves a dedicated design sub-document before Pattern 3 or tensor work starts.

2.2.2 Effect System Is the Hidden Critical Path

The correctness of MM2 lowering, snapshot isolation, optimization barriers, and the entire local search episode (Pattern 2) depends on the effect system. But the document does not specify the effect analysis itself — what is the effect language, how are effects inferred, what is the precision/conservatism tradeoff? This is the hidden critical path. Recommendation: prioritize writing the effect taxonomy and inference rules as a separate spec before Pattern 2.

2.2.3 Hot-Region Detection Is Unspecified

The human workflow (Section 12) assumes we know which regions are hot, but the mechanism — runtime profiling, sampling counters, manual annotation, cost-model-driven JIT thresholds — is not described. For the MVP this is fine (manual annotation), but the scalability story needs it. Suggestion: add a profiling instrumentation design as a parallel workstream once the match-kernel MVP proves out.

2.2.4 Space Revision Invalidation Under Concurrent Mutation

The CettaPack manifest checks space revision/epoch, but what happens to a *running* kernel when the space is mutated mid-execution? For read-only spaces this is moot, but for append-only or snapshot-isolated patterns, the invalidation semantics must be precise. Epoch-based revalidation at kernel entry is probably sufficient, but the document should state this explicitly.

2.2.5 Certificate Layer Risks Becoming Documentation Rather Than Verification

The MVP wisely emits certificate *skeletons*, but the gap between “data describing transformations” and “Lean-checked proof” is large. The five initial claims are the right starting set, but recommendation: make the Lean checker for claim 3 (native seeded rematch parity) the first proof milestone — it is the lynchpin of the entire PathMap+rematch strategy, and getting Lean to verify it early will either validate the approach or surface a semantic mismatch that would be catastrophic to discover late.

2.2.6 Missing Anti-Pattern: Space Schema Stability Across Profile Upgrades

One missing anti-pattern (Section A.9): “Do not assume space schema stability across profile upgrades.” A schema migration could silently invalidate cached CettaPacks; the loader must hash the schema, not just its name.

2.3 Overall Assessment

The architecture is sound and the “not just X” arguments against the three simpler plans are convincing. The MVP scope (one match site to C kernel with PathMap + rematch + manifest + fallback + tests) is well-chosen — small enough to build, large enough to validate every substrate layer. The harder patterns (local search episode, factor kernels, MM2 lowering) should be informed by what the MVP reveals about the CettaPack lifecycle, guard checking overhead, and PathMap index maintenance.

3 ProtomegaBot (OmegaClaw) Feedback on the CeTTa Proposal

3.1 Strengths Identified

1. **CettaPack manifest-guard pattern** is the single most important design choice. Having every specialized kernel carry its own preconditions (profile, space schema, revision policy, effect class) and fall back to generic evaluation on guard failure means the supercompiler can be aggressive without being dangerous. This mirrors how real JIT compilers work (deoptimize to interpreter on assumption violation) and is consistent with the fail-closed principle.
2. **Four optimization patterns well-chosen and correctly ordered.** Match-site specialization (Pattern 1) is the highest-ROI starting point because match is the hottest operation in most MeTTa programs and the PathMap-candidate-plus-native-rematch pattern is semantically clean. Saving backward chaining (Pattern 2) and PLN factor kernels (Pattern 3) for later is right — they are harder and benefit from having the match infrastructure stable first.

3.2 Technical Concerns and Suggestions

3.2.1 Space Revision Policy Needs More Specification

The manifest says `required_space_revision_or_epoch_policy` but Section 5’s kernel loads “the guarded space schema and revision” without specifying what happens under concurrent mutation during a kernel call. If a space is append-only, a snapshot-at-call-start is sufficient. If it is mutable, the kernel needs either a read lock or a snapshot isolation mechanism. CeTTa’s snapshot extension is mentioned (Section 2.4) but the interaction between snapshot isolation and the `effect: ReadOnly` manifest field is not spelled out. Recommendation: make the effect field distinguish `ReadOnly` (safe under concurrent append), `SnapshotIsolated` (safe with explicit snapshot), and `Mutation` (requires barrier).

3.2.2 Certificate Layer Risks Becoming a Bottleneck

Emitting LeaTTa/Lean certificates for every transformation is aspirationally correct but practically expensive. The MVP should start with data-only certificates (the `transform.cert` skeleton in A.5) and treat Lean checking as a later hardening phase, not a blocker for the first kernels. The document suggests this in A.5 (“emit certificates as data first, make the checker stricter over time”) but the main body could be clearer that certificate *checking* is not on the critical path for milestone 1.

3.2.3 Bag Multiplicity Handling Needs a Concrete Protocol

The document says “counts should be separate metadata” but does not specify where. If PathMap stores canonical support (distinct atoms) and the kernel reconstructs bag multiplicity, the count metadata needs to live somewhere — either in the PathMap index, in the kernel’s local state, or in a sidecar. Recommendation: keep it in the kernel’s local state during the call, reconstructed from the space’s own multiplicity records, never stored in PathMap itself. This keeps PathMap as a pure candidate-narrowing index.

3.2.4 Local Search Episode Is Underspecified Relative to Its Ambition

“Compact choice frames, trail/watermark rollback, scratch arena” describes a WAM-like machine. That is a substantial implementation effort. The document should acknowledge that Pattern 2

is closer to “build a mini-Prolog engine inside CeTTa” than to “generate a specialized kernel.” Consider whether PeTTa interop (calling out to PeTTa for the search episode and importing results) is a viable intermediate step before building a native CeTTa search machine.

3.2.5 No Mention of GPU Lowering Path for Tensor Presentation

Section 4 lists “tensor semiring contractions, batched sparse or dense kernels, possible GPU lowering” but the implementation roadmap (Section 14) does not include it. If PLN factor kernels (Pattern 3) are a target, GPU truth-value arithmetic could be a major win for large knowledge bases. Worth at least a placeholder in the roadmap.

3.2.6 Benchmarking Discipline

“Do not benchmark only against the generated kernel; always compare with generic CeTTa and PeTTa” is exactly right. Addition: also benchmark against HE (the reference evaluator) to catch profile-specific divergences that produce correct-but-different results.

3.3 Connection to OmegaHive 1.1

This supercompilation architecture fits naturally into the hive’s spawn-on-demand model. The CeTTa supercompiler itself could be a spawned worker (formalization worker template), with each compilation job as a bounded task contract: source span, profile, statistics, output artifact, done criteria. The CettaPack artifacts become experiment-ledger entries with reproducibility records. The Lean certificate checker could be a separate spawned worker using the Lean service from the hive infrastructure. The multi-geometry IR planner is essentially a “resource manager” for computational geometry — choosing the right execution presentation is analogous to choosing the right model for a task.

One thing the document does not address: **how supercompilation artifacts are cached and shared across hive agents**. If multiple agents compile the same hot match site, they should share the CettaPack artifact. This suggests CettaPack artifacts belong in the shared document/artifact volume (OmegaHive 1.1 Section 3.5), indexed by source hash + profile + space schema, with the memory curator handling dedup.

3.4 Additional Observations After Reviewing the v2 Background Document

The v2 document’s effect algebra (Section 4) is more mature than the CeTTa document’s effect summary. The CeTTa doc’s **effect: ReadOnly** manifest field should map directly to the v2 effect algebra’s **Read(resource, scope)** with commutativity rules. Specifically, the CeTTa manifest should carry the full effect class (not just “ReadOnly” vs “Mutation”) so the loader can verify commutativity with concurrent space operations. The v2 algebra already distinguishes **Read**, **Append**, **Write**, **Observe**, **Create**, **Delete** — CeTTa manifests should use the same vocabulary rather than collapsing to a coarse binary.

Also: the v2 document’s incremental statistics maintenance (Section 5.2) directly addresses the space-revision concern. The **ShouldReplan** algorithm detects when statistics drift warrants recompilation. CeTTa’s manifest guards should incorporate this: instead of a static revision check, the kernel should track a statistics-drift threshold and fall back to generic evaluation when drift exceeds the adaptive threshold. This makes the guard dynamic rather than static, which is important for append-only spaces that grow over time.

4 Cross-Document Comparison: CeTTa vs. MORK-Native Supercompiler v2

This section synthesizes ProtocosmoBot’s analysis of how the CeTTa proposal (July 2026) relates to its predecessor, the MORK-Native Supercompiler v2 (October 2025).

4.1 What CeTTa Preserves from the v2 Document

1. **Minimal core IR philosophy.** M-Core (v2 Section 3) maps directly to CeTTa’s residual MeTTa + operator schema approach. The node types (Sym, Var, Con, App, Match, Choice, Prim) are the same substrate, just rebranded.
2. **Effect algebra.** The commutativity relations (Algorithm 1) and sink-free semantics (Section 4.3) are the direct ancestor of CeTTa’s effect system (Section 2.4 in the CeTTa doc). The key insight — that monotonic multiset additions enable permanent caching, speculation without rollback, and append-only indices — survives intact.
3. **Canonical keys for termination.** The CanonicalPathSig / CanonicalKBSig / CanonicalEffectSig structures (Section 6.3) are the precursor to CeTTa’s CettaPack manifest guards. The subsumption relation (Algorithm 10) is the same idea: fold when a more general key subsumes a specific one.
4. **Bounded splitting with statistics.** Algorithm 9’s probability-guided branch selection with a 0.95 cumulative probability cap and catch-all for soundness is essentially what CeTTa’s Pattern 2 (local search episode) needs.
5. **Bisimulation for MM2 compilation correctness.** Section 9’s proof structure is retained in CeTTa’s certificate layer (Section A.5).

4.2 What CeTTa Changes or Supersedes

1. **Multi-geometry IR is new.** The v2 document commits to a single minimal core IR (M-Core) with domain-specific compilation to primitives. CeTTa’s key innovation — *deferred presentation choice* among C, PathMap/trie, factor graph, tensor, MM2, and residual MeTTa — is absent in v2. The v2 architecture is “compile everything to M-Core then supercompile.” CeTTa says “do not commit to a presentation until you have profiled.”
2. **No CettaPack / guarded artifact system in v2.** The v2 document has versioned indices (Section 7.2) but nothing like the CettaPack manifest with runtime guard checking and automatic fallback. The safety architecture is weaker — correctness depends on the supercompiler doing the right thing, rather than on the loader verifying guards at execution time.
3. **No PathMap / native rematch distinction in v2.** The v2 document treats `kb_query` as a monolithic primitive. CeTTa’s two-stage strategy (PathMap narrows candidates, native seeded rematch decides) is a refinement that does not appear in v2.
4. **Query planner is the primary optimization layer in v2.** In CeTTa, the query planner role is absorbed into the broader “planner chooses presentation” step. The cardinality estimation and effect-aware join ordering algorithms (Algorithms 2–6) are still relevant but become inputs to presentation choice rather than the optimization itself.

5. **No mention of Lean/LeaTTa certificates in v2.** Section 12 of v2 claims formal correctness properties but through traditional proof structures (bisimulation, induction). CeTTa’s plan to emit machine-checkable certificates and have Lean verify them is a significant upgrade in trustworthiness.

4.3 What the v2 Document Has That CeTTa Could Use

1. **The full effect algebra and commutativity algorithm.** CeTTa references an effect system but does not specify it. The v2 document’s Algorithm 1 is concrete and implementable. It should be pulled into CeTTa as the effect taxonomy spec (which ProtocosmoBot flagged as missing).
2. **Incremental statistics under monotonic growth** (v2 Section 5.2). The delta statistics maintenance and adaptive replanning algorithms are directly relevant to CeTTa’s hot-region detection (which was flagged as unspecified). The growth-rate tracking and drift-based replanning thresholds are a concrete answer to “how do we know when to recompile.”
3. **Evolutionary specialization gating** (v2 Section 8). The amortization analysis for fitness function specialization is a useful pattern that CeTTa’s Pattern 4 (factor kernels) could adopt for deciding when specialization pays off.
4. **Semi-naive KB saturation** (Algorithm 11). Concrete incremental saturation with delta tracking. Relevant if CeTTa’s Pattern 3 (active subgraph tensor extraction) needs to maintain consistent views of a changing space.

4.4 Overall Assessment of the Pair

The CeTTa document is the right evolutionary step — it generalizes from “compile to one IR and supercompile” to “choose the right presentation and guard it.” But the v2 document fills several gaps flagged in CeTTa: the effect algebra, the statistics machinery, and the incremental maintenance story. The two should be treated as companion documents, with CeTTa as the architecture spec and v2 as the algorithms-and-formal-semantics reference.

The v2 implementation roadmap (Section 10) is also more concrete than CeTTa’s MVP plan — Phase 0 through Phase 4 with clear deliverables. CeTTa’s MVP (one match site to C kernel) is a subset of v2’s Phase 2. Aligning them explicitly would help: CeTTa’s milestone 1 (hot match kernel) should map to v2’s Phase 0 (minimal core + query planner) plus the first part of Phase 2 (structural stepper on match sites).

The two documents together form a coherent design. The v2 document is the theory; the CeTTa document is the engineering. The gap between them is the implementation roadmap — v2’s Phase 0–4 roadmap is more abstract, while CeTTa’s Section 14 roadmap is more concrete.

5 Consolidated Suggestion List

This section provides a single-itemized list of all actionable suggestions from the discussion, tagged by contributor.

5.1 Specification Gaps to Fill

1. **Write the effect taxonomy and inference rules as a separate spec.** [ProtocosmoBot]
The effect system is load-bearing for Patterns 2–4 but unspecified. Use v2 Algorithm 1 as

the starting point.

2. **Create a dedicated tensor presentation design sub-document.** [ProtocosmoBot] The lowering story from MeTTa semantics to semiring contractions / GPU kernels needs concrete specification before Pattern 3 implementation.
3. **Specify hot-region detection mechanism.** [ProtocosmoBot] Add a profiling instrumentation design as a parallel workstream once the match-kernel MVP proves out. Consider v2’s incremental statistics (Section 5.2) and `ShouldReplan` algorithm as foundations.
4. **Spell out space revision invalidation semantics.** [ProtocosmoBot, ProtomegaBot] What happens to a running kernel when the space is mutated mid-execution? Epoch-based revalidation at kernel entry is probably sufficient. Make the effect field distinguish `ReadOnly`, `SnapshotIsolated`, and `Mutation`.
5. **Specify bag multiplicity protocol.** [ProtomegaBot] Where does count metadata live? Recommendation: kernel’s local state during the call, reconstructed from the space’s own multiplicity records, never in `PathMap` itself.
6. **Add GPU lowering placeholder to the roadmap.** [ProtomegaBot] If PLN factor kernels (Pattern 3) are a target, GPU truth-value arithmetic could be a major win. At minimum a placeholder in Section 14.
7. **Add anti-pattern: “Do not assume space schema stability across profile upgrades.”** [ProtocosmoBot] The loader must hash the schema, not just its name.

5.2 Architecture and Design Suggestions

1. **Use full v2 effect vocabulary in CeTTa manifests.** [ProtomegaBot] Instead of coarse “`ReadOnly`” vs “`Mutation`,” use v2’s `Read`, `Append`, `Write`, `Observe`, `Create`, `Delete` so the loader can verify commutativity with concurrent space operations.
2. **Make manifest guards dynamic, not static.** [ProtomegaBot] Incorporate v2’s statistics-drift threshold and `ShouldReplan` algorithm into manifest guards. Fall back to generic evaluation when drift exceeds the adaptive threshold. Important for append-only spaces that grow over time.
3. **Consider PeTTa interop as an intermediate step for Pattern 2.** [ProtomegaBot] Before building a native CeTTa search machine (which is closer to “build a mini-Prolog engine” than “generate a specialized kernel”), consider calling out to PeTTa for the search episode and importing results.
4. **Align CeTTa and v2 roadmaps explicitly.** [ProtocosmoBot] CeTTa’s milestone 1 (hot match kernel) should map to v2’s Phase 0 (minimal core + query planner) plus the first part of Phase 2 (structural stepper on match sites).

5.3 Verification and Certificate Suggestions

1. **Make Lean checker for claim 3 (native seeded rematch parity) the first proof milestone.** [ProtocosmoBot] It is the lynchpin of the entire `PathMap`+rematch strategy. Getting Lean to verify it early will either validate the approach or surface a semantic mismatch that would be catastrophic to discover late.

2. **Start with data-only certificates; treat Lean checking as later hardening.** [ProtomegaBot] The MVP should emit `transform.cert` skeletons and not block on Lean verification for the first kernels.

5.4 Benchmarking and Testing Suggestions

1. **Benchmark against HE (reference evaluator) in addition to generic CeTTa and PeTTa.** [ProtomegaBot] Catches profile-specific divergences that produce correct-but-different results.

5.5 Integration and Sharing Suggestions

1. **CettaPack artifacts should be shared across hive agents.** [ProtomegaBot] Indexed by source hash + profile + space schema in the shared document/artifact volume, with the memory curator handling dedup.
2. **CeTTa supercompiler as a spawned worker in OmegaHive.** [ProtomegaBot] Each compilation job as a bounded task contract. Lean certificate checker as a separate spawned worker. Multi-geometry IR planner as a resource manager for computational geometry.

6 Participant Attribution Summary

Contributor	System	Contributions
ProtocosmoBot	ZeroBot / OpenClaw	CeTTa proposal review (6 strengths, 6 concerns); cross-document comparison (5 preserved, 5 superseded, 4 reusable, overall assessment); missing anti-pattern
ProtomegaBot	OmegaClaw	CeTTa proposal review (2 strengths, 6 concerns); OmegaHive 1.1 integration analysis; v2 background document observations (effect algebra mapping, dynamic guards); CettaPack sharing suggestion
Oruzibot	—	Mentioned in the discussion request; no CeTTa-specific feedback recovered from available session logs
Ben Goertzel	—	Authored both source documents; shared v2 as background context; requested this compilation

Note: Oruzibot was tagged in the original request for feedback. If Oruzibot provided feedback through a channel not captured in the available session logs, it is not included here. Ben may wish to check directly with Oruzibot for any additional CeTTa feedback.

7 Recommended Next Steps

1. **Fill the effect taxonomy gap** using v2 Algorithm 1 as the starting point. This is the highest-priority specification gap because Patterns 2–4 all depend on it.

2. **Build the match-kernel MVP** (Pattern 1) as the first concrete deliverable. It exercises every substrate layer (PathMap candidates, native rematch, C kernel generation, CettaPack loading, manifest guards, fallback) without requiring the harder patterns.
3. **Prototype the Lean checker for native seeded rematch parity** as the first certificate milestone. This validates the lynchpin semantic claim early.
4. **Align the CeTTa and v2 roadmaps** explicitly so implementation milestones map to formal-semantics milestones.
5. **Design the profiling/hot-region detection mechanism** as a parallel workstream once the MVP reveals CettaPack lifecycle characteristics.
6. **Create the tensor presentation design sub-document** before starting Pattern 3 work.